

Appendix A

An Overview of Extreme Programming

This appendix serves as a brief introduction to the main ideas of Extreme Programming (XP). If you are already familiar with XP you can safely skip this appendix. If not, please use this appendix as an introduction to XP and then proceed to one of the fine books that explain XP in detail.¹

We'll look first at the people (or roles) involved in an XP project. Next we'll look at the twelve main practices of XP. We'll conclude by considering the values of an XP team.

Roles

The XP customer role is responsible for writing stories, prioritizing stories, and writing and executing tests that demonstrate that stories were developed as expected. The XP customer may be a user of the system being built but that is not necessary. If not a user, the XP customer is often a product manager, project manager or business analyst.

On some projects the customer role may actually be filled by a customer team, which would be composed of multiple highly interested individuals. The customer team will often include testers who assist with creating the acceptance tests. When a project has multiple customers, it is important that they speak with one voice. They can achieve this in many ways, but most commonly by designating one individual on the customer team as the first among equals.

1. In particular see *Extreme Programming Explained: Embrace change* (Beck 2000), *Extreme Programming Installed* (Jeffries, Anderson, and Hendrickson 2000), or *Extreme Programming Explored* (Wake 2002).

The XP programmer role encompasses a broad range of technical skills. XP projects tend not to draw distinctions between programmers, designers, database administrators, and so on. All programmers are expected to work as a team and to share many responsibilities that might otherwise be assigned to specific individuals on a non-XP project. Almost all processes expect programmers to unit test their code; XP takes this expectation seriously and programmers are expected to develop automated unit tests for everything they write.

Finally, many XP teams benefit from the use of an XP coach and possibly a project manager. The roles are sometimes combined within a single individual. The coach is responsible for monitoring the team's use of the XP practices and gently nudging them back on track when they stray. The project manager is more leader than manager and is responsible for sheltering the team from bureaucracy and removing as many obstacles as possible.

The Twelve Practices

Extreme Programming is characterized by the twelve practices described in Kent Beck's original "white book" (Beck 2000). If you choose to try XP on a project, you are highly encouraged to adopt all of the practices. XP's twelve practices are highly synergistic and interdependent. Practices support and enable each other. For example, the practice of refactoring is made easier by the practices of pair programming, simple design, collective ownership, continuous integration, and testing. The twelve practices are not a random collection of good ideas from which an XP team is allowed to pick their favorites. After becoming experienced with XP, you may choose to drop or alter a practice but you really should delay customizing XP until becoming experienced with standard XP.

In this appendix we will consider the following twelve XP practices:

- small releases
- the planning game
- refactoring
- testing
- pair programming
- sustainable pace
- team code ownership

- coding standard
- simple design
- metaphor
- continuous integration
- on-site customer

Small Releases

XP projects progress in a series of iterations, each of which is each typically one to three weeks long. Features, as described by user stories, are fully delivered within a single iteration. A team is not allowed to deliver half of a feature. Similarly, a team is not allowed to deliver the full feature but at half the quality. By the end of each iteration the team is responsible for delivering working, tested code that can immediately be put to use.

At the start of the project the team selects an iteration length that it will use for the duration of the project. The iteration length is typically one or two weeks and never longer than four. The team should select an iteration length that is as short as possible yet that will still deliver tangible value to the business. When undecided between two durations, choose the shorter one.

Iterations are firm timeboxes. A team cannot reach the planned last day of an iteration and decide they need two more days. The iteration ends on the scheduled day. The amount of work the team does (but not the quality of that work) is adjusted to accommodate the iteration.

The Planning Game

The “Planning Game” is the XP name for release and iteration planning during which the developers and customer collaboratively make predictions about the future. Prior to initiating planning, the customer has written user stories on note cards and the developers have estimated the cost or magnitude of each story and have written the estimate on the story card.

To start planning, the developers estimate how much work they can complete in an iteration of the length selected for the project. The customer then looks over all of the story cards and selects her top priority stories for inclusion in the first iteration. She is allowed to select stories that sum to but do not exceed the amount of work the developers estimate they can do in the iteration. Once the first iteration is full of work, the customer moves onto selecting stories for the second and subsequent iterations.

After some number of iterations, the customer decides that enough stories have been placed into iterations that collectively they define a release. The release plan almost certainly does not accurately reflect what will be developed and in what order. The release plan is a hypothesis about how development may proceed but it will be updated at the start of each iteration as priorities change, as more becomes known about the rate of progress by the team, and as the developers learn more about the true expected costs of each story.

Prior to the start of each iteration, the team and customer plan the iteration. This involves selecting the highest priority stories that can be completed in the iteration and then identifying the specific tasks necessary to complete the story.

Refactoring

Refactoring (Fowler 1999; Wake 2003) refers to the restructuring or rewriting of code so as to improve the code without changing its external behavior. Over time code can get ugly. A method designed for one purpose is changed slightly to handle a special condition. Then, since it already handles that special condition, it's changed again to handle another special condition. And so on until the method becomes too fragile for further modification.

XP advocates constant attention to refactoring. Whenever a programmer makes a change to code that should be refactored, she is required to refactor it. She's not encouraged to refactor it; she's required to refactor it. In this way code avoids the slow, sometimes hard to detect decay that eventually results in obsolescence.

Refactoring is one of technique used by XP to replace upfront design. Rather than spending time upfront thinking through a system in advance of coding it, and therefore taking guesses at some aspects of its behavior, XP systems are refactored and kept in a state that perfectly meets known, implemented requirements.

Testing

An exciting practice of XP is its focus on testing. On an XP project the developers write automated unit tests and the customers write acceptance tests, which are often automated either by the customers themselves or with some help from the developers. Many XP developers have really found the benefits of early and frequent testing. Further, traditional resistance by developers to testing has gone down because XP unit tests are typically automated by writing test code that exercises the operational code—that is, even while testing they are programming.

In traditional development tests are written after the code (if they're written at all). This becomes a problem because once code is written and appears to work, it can be human nature not to push too hard on the code. So, many developers push gently on their code and call it tested. (I know this: I used to be one of them.) XP changes this and puts the tests right up front in a practice called *test-driven development* (Beck 2003; Astels 2003).

In test-driven development, tests are written before the code. Developers follow a short (minutes, not hours) cycle of test-code-test-code and so on. They follow a rule that no operational code may be written except in response to a failing test. So, they write a test that fails. The program is run to verify that the test fails. Only then does a programmer write the code that makes the program pass the test.

Test-driven development guarantees that code will remain well factored and testable. It also leads to more maintainable code since code is effectively in a maintenance mode right from the start.

In addition to programmer unit testing, customer tests are an important part of XP. For each story, the customer is responsible for defining a series of tests that are used to determine if the story was developed in accordance with the customer's expectations and assumptions. In many ways, these customer-written acceptance tests replace the requirements documents of a waterfall process.

Pair Programming

One of XP's more controversial practices is pair programming. Pair programming refers to two programmers sharing one keyboard and one monitor but using their two brains to write code. While one programmer is typing at the keyboard (and mentally thinking a few lines ahead in his code), the second programmer is watching the code develop and thinking more broadly about where the code may lead and what problems it may encounter. Pairs switch roles and partners often.

While pair programming may sound tremendously inefficient, Alistair Cockburn and Laurie Williams (2001) have studied it and found that not to be the case. They found that for a 15% increase in total programming time, pair programming leads to:

- lower defect counts
- less code is written to solve the same problem
- problems being solved more quickly
- more people understand each piece of code

- an increase in developer job satisfaction

Pair programming is important to XP because so many of the other XP practices require discipline. It requires a tremendous amount of discipline to refactor every time you notice poorly structured code or to always write tests before writing operational code. Without a pair it can be far too tempting to skip a refactoring or a test with the thought of “Just this once...”

Sustainable Pace

XP teams are encouraged to work at a sustainable pace. The belief is that an XP team moving at a consistent but brisk pace will achieve more over a period of time than will a team working at a pace they cannot sustain over a long period. This does not mean that an XP team works exactly forty hours each week and then heads for home. It is up to the team to determine their sustained pace, and it is likely to be different for different members of the team.

Pair programming and test-driven development are effective because they focus the minds of the pair very intensely on the code they are creating. Few people are capable of maintaining this level of intensity for extended periods. A team will typically devote around six hours per day to pairing and spend the remainder of the day in other activities.

An XP coach is responsible for monitoring the team for burnout. If the coach senses a team is burning out she will help the team shift back to working at a sustainable pace.

Team Code Ownership

It has been common among non-XP teams for individual developers to “own” or assume full responsibility for portions of a system’s code. In this way each portion of a system would be owned by one developer—at least until a developer moves onto another project and her code is left without an owner. This view toward code ownership also leads teams to make comments such as “We can’t change the billing source code until Eli gets back from vacation.” Further, if a developer is talked into changing the code while Eli is on vacation, Eli will probably be angry about changes made to “his code” when he returns.

XP teams take a completely different approach to code ownership: All code is owned by everyone. Under this model of team ownership, any pair of developers can change any code. In fact, because of the refactoring practice, pairs are expected to change code they didn’t write.

Individual ownership is practiced to ensure coherent design and to keep all responsibilities of a module balanced. In XP, this burden is borne by test-driven

development. A strong suite of unit tests ensures that changes do not introduce unanticipated side effects.

Coding Standards

Because XP teams collectively own their source code, it is important that they follow a coding standard. The coding standard lays out the main rules and conventions team members will follow when writing code: how will variables and methods be named? how will lines be formatted? and so on.

A small, close-knit team may be able to get by without a written, formalized coding standard. They can build and share standards through team folklore. Beyond a handful of developers, most teams will benefit from writing down their coding standards but still keeping them as brief and essential as possible.

Simple Design

XP teams pursue a goal of having the simplest possible design that delivers the features a customer needs. Kent Beck (2000) defines four constraints that indicate when a design is the simplest it can be:

1. The operational code and the test code fully convey the programmer's intent for the behavior of that code.
2. There is no duplicated code.
3. The system uses the least number of classes.
4. The system uses the least number of methods.

Metaphor

XP teams support the quest for simple design by finding a metaphor that can be used for the whole system. The metaphor provides a frame of reference for how they think about the system. For example, on one project our metaphor was that the system was like a chalkboard and that various parts of the system could write on the chalkboard. When the user was done with the system she would either save the contents of the chalkboard or erase them. This greatly simplified way of thinking about the system helped us by giving us a convenient, simple way of thinking about the behavior of the system.

Continuous Integration

I got involved in a discussion recently with an executive at one of the largest eCommerce companies. He told me that integrating the work of multiple developers was the largest problem for most software development teams. He liked to have his teams integrate their software once a month so they could avoid the bigger problems of integrating less frequently. I asked him what would happen if his teams instead integrated on a daily basis.

XP teams know the answer and they integrate at least daily. We learned long ago about the benefits of a daily build and smoke test (Cusumano and Selby 1995). XP teams have taken this to the point where code is integrated more or less continuously. For example, a developer completes a small change, she checks the change into the source code repository where a process notices the change and initiates a full build. When the build is complete a suite of automated tests is run. If any of the tests fail, the developer is emailed and told about the failure. Integration problems are fixed one at a time in extremely small batches as soon as they occur.

On-Site Customer

It used to be common for a customer to write a requirements document, throw it over a wall to the programmers who would write the code and then throw the system over another wall to some testers. With XP the walls are gone and the customer is expected to sit with and be part of the development team. The customer writes the stories and the acceptance tests and also is on hand to answer questions as soon as they arise.

The on-site customer is essential to successful use of the user story approach because of the many conversations that must occur between the customer and the developers. If the customer is not on site, delays will disrupt the predictable progress of the XP team.

XP's Values

In addition to its practices, XP advocates four values: communication, simplicity, feedback and courage. XP values communication but not all modes of communication are equal. Most desirable is face-to-face communication where we can talk, respond, gesture and draw on a whiteboard. Less desirable are written documents. XP stresses communication through practices such as pair programming.

Simplicity is a value of XP teams because it keeps their focus on creating a solution to the problem faced today, not the problem anticipated tomorrow. XP teams do not architect a system with support for features beyond those that are necessary for developing the features of the current iteration. They remain constantly focused on doing the simplest thing that could possibly work.

XP teams value feedback, and the more immediate the feedback the better. XP developers give and get feedback during pair programming when one developer points out a potential problem to her pair. They get feedback from the automated tests they run so often. They get feedback from their continuous (or at least daily) integration process. Customers are part of the team, even sitting in the same space with the developers, and provide feedback through constant interaction with the team and through the acceptance tests they write.

Finally, XP teams value courage. For example, they have courage to refactor their code (because they have automated tests to back up that courage). They have courage to proceed without an overall master architecture because they will use a metaphor and maintain a simple design through refactoring and test-driven development.

The Principles of XP

In addition to its values and practices, XP can be characterized by its five basic principles: rapid feedback, assuming simplicity, incremental change, embracing change, and doing quality work (Beck 2000). Since the introduction of XP, a debate has raged over whether a team can be doing XP if they doing only eleven of the original twelve practices. Is a team that doesn't pair program doing XP? Is a team that pursues a simple design but starts with a few weeks of modeling doing XP?

I think the answer is yes. These teams are doing XP if they are living by the principles of XP. A team that:

- provides rapid feedback to its customers and learns from that feedback
- favors simplicity and always attempts a simple solution before moving to a more complex one
- improves the software through small, incremental changes
- embraces change because they know they are truly adept at accommodating and adapting

- and, insists that the software consistently exhibits the highest level of quality workmanship

must certainly be doing XP, even if they missing a practice or two.

Summary

- The XP customer role is responsible for writing stories and acceptance tests for each story, and sits with the development team.
- On an XP project the distinction between programmer and tester is blurred. Programmers write unit tests of their own code; testers program automated acceptance tests.
- XP projects include a coach and possibly a separate project manager who are responsible for guiding the team and removing obstacles from its way.

Extreme Programming involves the following practices:

- small releases
- the planning game
- refactoring
- testing
- pair programming
- sustainable pace
- collective code ownership
- coding standard
- simple design
- metaphor
- continuous integration
- on-site customer

And has these values:

- communication
- simplicity

- feedback
- courage

And has these key principles:

- rapid feedback
- assume simplicity
- incremental change
- embrace change
- quality work

This page intentionally left blank